`scripts/advance-all-submodules.sh` — User Guide

Last verified: 2026-08-26 (re-verified line-by-line against the current script after the T054 round-5 review; `CM-SCRIPT-DOCS-SYNC`)
Inheritance: HelixConstitution §11.4.18 (script documentation mandate) + Lava §6.T.3 (no force push) + §6.W (2-mirror scope) + the Decoupled Reusable Architecture rule's submodule-pin policy and its **Automated Pipeline Pin-Advance Path** conditions (A)–(F), as amended by T048 for this pipeline

Overview

Advances every submodule of a repository to its own upstream's latest commit, one submodule at a time, and writes a **Submodule Advance Record** per submodule describing exactly what happened.

It implements `specs/002-build-test-distribute-pipeline/research.md` **R-005**'s 7-step ordering verbatim:

1. `git fetch --all` in the submodule
2. compare the currently-pinned commit to the remote default branch's HEAD
3. identical → **no-op** (spec Edge Case: "no newer commit available")
4. different → check the remote HEAD out into the submodule's working tree
5. rebuild + re-run the affected test categories against the advanced state; **on failure the advance is discarded** (the prior pin is checked back out) rather than recording a pin bump that broke the build
6. if the submodule carries local, uncommitted modifications, commit and push them to its own upstream(s)
7. `git add <submodule-path>` in the **parent** repository to record the new pin

The rebuild-and-test-*before*-recording-the-pin ordering is the whole point: it makes the "advance breaks the build" edge case mechanically enforced instead of a documentation-only promise.

Usage

```
# Real use: whole repository, inside a pipeline run
LAVA_PIPELINE_RUN_ID=2026-08-21T14-30-00Z scripts/advance-all-
        submodules.sh

# Explicit repository path (also how the hermetic test points it
        at fixtures)
scripts/advance-all-submodules.sh /path/to/repo

# There is NO way to carry a submodule's uncommitted work forward.
        R-005
# step 6 and the --publish-local-modifications flag that gated it
        were REMOVED
# on 2026-08-26; passing the flag is exit 2 naming the removal. An
        unclean
# submodule working tree is refused, full stop. Commit, stash or
        remove the
# work yourself — with a human reading the diff — and re-run.

# Only for disposable fixtures: permit a remote that is a
        filesystem path.
scripts/advance-all-submodules.sh --allow-local-path-remotes /tmp/
        fixture-repo

# One submodule only, with a custom verification step
LAVA_PIPELINE_RUN_ID=2026-08-21T14-30-00Z \
LAVA_ADVANCE_SUBMODULES="submodules/auth" \
LAVA_ADVANCE_VERIFY_CMD='scripts/ci.sh --changed-only' \
  scripts/advance-all-submodules.sh
```

Inputs

Input	Required	Meaning
`$1` (positional)	no	Parent repository path. Default: `git rev-parse --show-toplevel`.
`--allow-local-path-remotes`	no	Permit a submodule remote that is a plain filesystem path. Without it such a remote is refused (`REFUSED_FOREIGN_UPSTREAM`) like any other out-of-scope upstream. Exists so the hermetic suites can reach the fetch path at all — a filesystem path can be another real repository on this machine, or an NFS/SMB/sshfs mount that reaches another machine while naming no host, and the script FETCHES from every configured remote, so it is an object-injection surface.

Input	Required	Meaning
`--publish-local-modifications`	—	**REMOVED 2026-08-26.** Accepted by the parser only so that passing it is a loud `exit 2` naming the removal, never a silently-ignored argument and never a fall-through to a different mode. See "R-005 step 6 was removed" below.

The remaining switch is a **command-line flag and deliberately not an environment variable.** It relaxes a safety boundary, so the question that decides its shape is whether an unattended pipeline run could pick it up without anybody having asked for it in that run. An environment variable can — it is inherited wholesale from whatever the invoking shell exported, potentially hours earlier in an unrelated test session, and it survives every `bash script.sh` layer in between. `argv` cannot: a phase script invokes with a fixed argument list, so a flag has to be typed at the invocation site, in the run that uses it. Root `CLAUDE.md`'s Automated Pipeline Pin-Advance Path condition (D) draws the same line for force-pushing — an approval "an unattended pipeline cannot obtain and therefore may never assume". The environment is a channel through which it could assume one. | `LAVA_PIPELINE_RUN_ID` | conditionally | The pipeline run this advance belongs to. Required unless **both** `LAVA_ADVANCE_RECORD_DIR` and `LAVA_ADVANCE_VERIFY_CMD` are supplied — without a run id the script cannot honestly claim to have rebuilt-and-tested anything, since `phase-01-build.sh`/`phase-02-test.sh` both require a run id with an existing `report.json`. **Validated against `^[A-Za-z0-9._:-]+$`, and separately rejected when it is `.`, `..` or any run of dots**; anything else is exit 2 with nothing attempted. (The character class alone still admitted the only two path-navigation tokens it could. No `/` can appear so `..` cannot walk out of `.lava-ci-evidence`, but it does silently relocate a run's records out of its own run directory into the shared `pipeline-runs/` parent, where the next run's records land on top of them.) It is a path component of the record directory and it is handed to the verify step, so an unvalidated value was both a traversal vector and a command-injection vector — see the 2026-08-26 section. | | `LAVA_ADVANCE_RECORD_DIR` | no | Where Submodule Advance Records are written. Default: `.lava-ci-evidence/pipeline-runs/<run_id>/submodule-advances` (per `data-model.md`). | | `LAVA_ADVANCE_VERIFY_CMD` | no | The R-005 step-5 rebuild-and-test command, run once per advanced submodule via `bash -c`, with `$1` = the submodule's absolute path, `$2` = the parent repository root and **`$3` = the run id**. Zero exit ⇒ the advanced state still builds and passes. Default: `phase-01-build.sh "$3" "$2" && phase-02-test.sh "$3" "$2"`. The run id is passed **positionally**, never interpolated into the command text. | | `LAVA_ADVANCE_SUBMODULES` | no | Whitespace-separated allow-list of submodule paths (as they appear in `git submodule status`). Default: every submodule. **Every token must name a**

submodule this repository actually has — a token that matches nothing is exit 2, not an empty-but-successful run. It can only ever NARROW a run, and it can never widen the governance allow-list. |

Nothing is hardcoded (§6.R): the repository path, submodule set, remote name, and default branch name are all resolved at runtime from git itself — the default branch in particular comes from the remote's own `HEAD` symref (`git ls-remote --symref`), because this project's submodules do not all share one default branch name.

Outputs

One JSON file per submodule at `<record-dir>/<sanitized-submodule-path>.json`, conforming to `specs/002-build-test-distribute-pipeline/contracts/submodule-advance-record.schema.json`:

```
{
  "submodule_name": "submodules/auth",
  "old_commit": "32a80e0a…",
  "new_commit": "9f1c2b47…",
  "local_modifications_pushed": false,
  "parent_pin_updated": true,
  "outcome": "ADVANCED"
}
```

`outcome` is one of:

Outcome	Meaning	`parent_pin_updated`
`NO_NEWER_COMMIT`	Pin already equals the upstream default branch HEAD; nothing was done (step 3). `old_commit == new_commit`.	`false`
`ADVANCED`	Fetched, checked out, verified, and staged the new pin **in the parent index**. Nothing was committed and nothing was pushed — this script does neither, on any path.	`true`

Outcome	Meaning	`parent_pin_updated`
`REJECTED_BREAKING_CHANGE`	The advanced state failed the step-5 rebuild-and-test, or could not be checked out. Advance discarded; prior pin restored.	`false`
`REJECTED_PUSH_CONFLICT`	**RETAINED in the schema but currently UNEMITTABLE.** No code path in this script issues a `git push`; the only pushing code it ever had was R-005 step 6, removed 2026-08-26. The value stays in the enum because root `CLAUDE.md`'s Automated Pipeline Pin-Advance Path condition (E) names it explicitly as one of four outcomes, and a schema that cannot express a value the governing document names is a worse disagreement than a value nothing currently emits. Removing it needs an operator amendment to condition (E).	`false`
`REFUSED_GOVERNANCE_DENY`	The submodule carries no standing operator authorization, so	`false`

Outcome	Meaning	`parent_pin_updated`
	its upstream was never contacted — nothing was fetched, checked out, verified, committed or pushed for it. (The console line used to say "NOT examined" while the summary counted the same submodule among the "N submodule(s) examined"; both senses were defensible and the two lines contradicted each other, which is worse than either.) `old_commit == new_commit == the current pin`.	
`FAILED_PRECONDITION`	A precondition for examining or advancing it could not be established: uninitialized submodule, working tree not sitting on the pinned commit, **an UNCLEAN working tree** (there is no flag that overrides this), `git status` failure, no remote configured, failed fetch, unresolvable remote default branch, or the fetched object	`false`

Outcome	Meaning	`parent_pin_updated`
	absent after a successful fetch. Nothing was advanced, and for the unclean-tree case the upstream was never contacted at all.	
`REFUSED_NOT_FAST_FORWARD`	The remote default branch HEAD does not contain the pinned commit, so moving the pin would DROP pinned work. Nothing was checked out.	`false`
`REFUSED_FOREIGN_UPSTREAM`	A configured remote falls outside the own-org GitHub/ GitLab scope condition (C) permits. Nothing was fetched or advanced. The script FETCHES from every configured remote, so an out-of-scope one is an object-injection surface whose objects would end up under the pin staged for every clone.	`false`
`REJECTED_PARENT_STAGING_FAILED`	The advance completed but the parent index could not be staged, or it staged something other than the commit	`false`

Outcome	Meaning	`parent_pin_updated`
	this run measured.	

Four outcome values were REMOVED on 2026-08-26, together with R-005 step 6, because each named a cause that could arise ONLY while committing or pushing a submodule's local work, and no code path can now produce any of them: `REJECTED_VERIFY_MUTATED_LOCAL_WORK`, `REJECTED_SUBMODULE_STAGING_FAILED`, `REJECTED_SUBMODULE_COMMIT_FAILED`, `REJECTED_COMMIT_SCOPE_EXCEEDED`. A contract value nothing can emit describes a writer that does not exist, which is the same defect as a missing value in the other direction. **Zero Submodule Advance Records existed at rest at the time of removal — measured, not assumed: no `submodule-advances` directory exists anywhere under `.lava-ci-evidence`** — so no record at rest carries a value the enum no longer admits. The writer's whitelist and the schema enum are the same nine values, and the schema's `allOf` list is exactly that enum minus `ADVANCED`.

`local_modifications_pushed` is now `"const": false` in the schema and is emitted as the literal `false` by the writer, which no longer accepts it as a parameter. The field is retained rather than dropped because `data-model.md` and condition (E) both name it, and a consumer must not have to branch on its presence.

The record's own writer enforces the schema's `allOf/if-then` invariant, stated as the general rule: **`ADVANCED` is the only outcome that may claim the parent pin moved, and it must claim it.** Both directions are checked, so neither a refusal claiming an advance nor an advance claiming no pin move can be emitted.

Every submodule the script processes leaves a record — root `CLAUDE.md`'s Automated Pipeline Pin-Advance Path condition (E). Exactly one condition still cannot be recorded, and it is honest rather than an omission: a parent index whose gitlink for the submodule is unreadable. The schema requires a commit sha and none was ever established, so there is nothing to write that would not be invented; it is reported on stderr, counted, and reflected in the exit code.

One further file, written only when the repository genuinely has no submodules: `<record-dir>/_corpus.json`, with `"outcome": "CORPUS_EMPTY_CONFIRMED"` and the three independent readings that establish it (`.gitmodules` declares 0, `git submodule status` enumerated 0, the parent index holds 0 gitlinks). It is deliberately **not** a Submodule Advance Record and does not conform to that schema — it names no submodule, because there is none to name. Its leading underscore cannot collide with a record filename: `_advance_sanitize_name` strips leading underscores, so no

submodule path can ever sanitize to `_corpus`. It exists because `exit 0 + 0 advanced` on a console is indistinguishable, to `phase-07-closure.sh`, from "everything was already current" — an empty corpus has to be evidence at rest, not a line that vanishes with the terminal.

The run's summary now also accounts for governance refusals in their own bucket (`N refused for want of a standing operator authorization`). They previously fell into none of `advanced` / `already current` / `rejected/failed`, so the outcome line of a run in which a submodule had been refused read exactly like the outcome line of a run in which everything was already current. The script asserts before exiting that the buckets account for every submodule it says it examined.

Aggregating these records into the run's `report.json` (`submodule_advances[]`) is `phase-07-closure.sh`'s job (T055), not this script's.

Exit codes

Code	Meaning
`0`	Every submodule ended `ADVANCED`, `NO_NEWER_COMMIT`, or was refused for want of a standing operator authorization **on its own path** (`REFUSED_GOVERNANCE_DENY` via the allow-list). The last of those is exit 0 **deliberately**, and it is not an oversight — see "A path-level governance refusal is exit 0, on purpose" below.
`1`	At least one submodule was rejected or refused for a reason that means something is wrong: a breaking change, an **unclean working tree** (there is no flag that overrides this), an out-of-scope upstream, an authorized PATH whose upstream names a repository nobody authorized, an upstream identity that could not be read at all, a governance certification that examined zero identities, a parent index that could not be staged, or a precondition that could not be established. **A caller MUST treat exit 1 as stop-and-page-a-human, never as retry** — see "Exit 1 is sticky" below.
`2`	Usage/configuration error: bad repo path; missing run id with no overrides; a run id outside `^[A-Za-z0-9._:-]+$` or equal to `./..`; an unknown option or a second positional path; a record directory that cannot be created **or that exists and cannot be written to**; a scratch working directory `mktemp -d` refused to create; an `LAVA_ADVANCE_SUBMODULES` token naming no submodule this repository has; a run that somehow examined zero submodules; **a corpus `.gitmodules` declares that the enumeration did not cover** (whether it enumerated

Code	Meaning
	none of them or only some — see the 2026-08-26 round-5 section); a run whose outcome buckets account for fewer submodules than it says it examined; **an ambient git environment variable that could not be removed**; **a safety switch, the governance allow-list, or the identity boundary that does not reflect what this file declares** (something in the invoking environment is holding the name — `BASH_ENV` is the known channel and it IS set on the operator's host); or **a hardened object reader that could not prove it ignores `refs/replace/*`**. Nothing was attempted.

Exit 1 is sticky — a caller must page a human, never retry

A refusal that happens **after** the step-4 checkout can leave the submodule on that rejected commit: `restore_pin` uses a plain `git checkout --detach`, which git refuses when untracked working-tree files would be removed, and forcing is the flag family this script declines on principle (§6.T.3). Nothing is published, the parent pin is untouched, and the run says so loudly (`THESE SUBMODULES ARE LEFT ON A REJECTED COMMIT`). Since the 2026-08-26 removal of R-005 step 6 there is no rescue ref, because there is no commit of this script's making for one to hold — and a submodule carrying the operator's own uncommitted work never reaches the checkout at all.

The consequence a caller must plan for: **every subsequent run for that submodule fails closed** with `FAILED_PRECONDITION` ("working tree is at X but the parent pins Y ... REFUSING"), and prints the exact restore command. That is the correct direction — the leftover is never mistaken for a clean starting point — but it is a **human-intervention requirement**, not a transient error.

`scripts/pipeline/phase-07-closure.sh` (T055) MUST therefore treat exit 1 from this script as **stop and page a human**. Retrying it re-runs a refusal that cannot clear itself, and an unattended pipeline is **blocked, not degraded** until an operator runs the printed `git -C '<submodule>' checkout --detach <pin>`.

Side effects

- **In each submodule**: `git fetch`, a detached-HEAD checkout, and — only when local modifications exist — a commit plus a fast-forward push to every configured remote (§6.W: GitHub + GitLab).
- **In the parent repository**: `git add <submodule-path>` only. It **never commits and never pushes the parent** — that is

`phase-07-closure.sh`'s job, deliberately separated so a review gate sits between "pins staged" and "pins pushed".

Safety properties

The safe-operating precondition — a clean submodule working tree

Every irreversible thing this script can do sits behind one condition. With a **clean** submodule tree it fetches, checks out, verifies and stages a pin in the parent index: all local, all undoable, nothing leaving the machine. With an **unclean** tree the same run becomes a *publisher* — it commits what `git status -uall` reported and pushes it to that submodule's own upstream default branch, which §6.T.3 forbids undoing by force-push.

Until the 2026-08-26 re-review that condition was neither checked nor stated; the reviewer approved a run "today" only on the measured fact that `submodules/helixqa` happens to be clean, and said plainly that the safety "rests on a precondition the script neither checks nor states". It is now **both**:

- a clean tree is announced per submodule, naming what it rules out;
- an unclean tree is **refused** (`FAILED_PRECONDITION`), listing the exact paths, **with no opt-in of any kind** — the flag that used to authorise them was removed on 2026-08-26.

The refusal is the deliberate default, not the timid one. A single stray editor swapfile left in a submodule was demonstrably committed and published to another repository's default branch, unattended, exit 0, outcome `ADVANCED` — the `-uall` capture is total, so *any* untracked byte armed step 6, and step 6 published. The refusal happens **before the fetch**, so an unclean submodule's upstream is never even contacted and the operator's bytes are never touched: the verify command does not run, and there is nothing to rescue because nothing was moved.

R-005 step 6 was removed (2026-08-26)

What was removed: 607 lines implementing R-005 step 6 — staging the operator's captured paths, fingerprint and tree comparison, submodule index and `HEAD` checks, commit construction, five publish guards (PARENTAGE, TREE, OBJECTS, HISTORY, MESSAGE), push-time destination re-certification, the per-mirror push loop, mirror-divergence reporting and rescue-ref preservation — together with the `--publish-local-modifications` flag that was its only entry, five now-dead helper functions

(`save_rescue_ref`, `remote_default_branch`, `_path_fingerprint`, `_captured_tree`, `_push_destinations_in_scope`) and four outcome values.

Why. Five review rounds found **twelve** distinct, fixture-proven ways for content no guard had read to reach another repository's default branch through step 6. The count rose at every round — 3 → 5 → 7 → 9 → 10 → 11 → 12 — and every round confirmed the previous round's fixes while finding more. Round 3 stopped enumerating *routes* and began guarding the *artefact*; rounds 4 and 5 then enumerated *projections* of the artefact, and the eleventh was a projection nobody had listed: 4000 bytes of payload carried in the commit object's own author-name header, which reached both mirrors with all five guards green.

The convergent instrument was tried and **failed**: `git commit-tree` inherits `i18n.commitEncoding`, so the expected commit is built in the same adversary-controlled config context as the real one and the poison cancels out on both sides — byte equality PASSED while attacker bytes were published. Guarding had provably stopped converging.

What it cost, measured rather than assumed. Step 6 had no caller (`scripts/pipeline/phase-07-closure.sh` did not exist, and T055's design states the closure phase never passes the flag); FR-000 already makes the pipeline refuse to start on a dirty tree, using `--ignore-submodules=none` precisely so submodule dirt is visible, so in a pipeline run a dirty submodule is a precondition failure rather than an input; and all 61 recursive submodules were clean at the time of removal. The operator's own `git commit` handles the case better, with a human reading the diff.

The property this buys. This script now issues **no `git push` at all** — not to a submodule's upstream, not to the parent's mirrors, not on any path, with or without any flag — and creates **no commit**, in a submodule or in the parent. §6.T.3's prohibition on history-overwriting pushes is satisfied by construction rather than by careful flag selection. All twelve escapes are unreachable because the code they lived in does not exist.

What this does NOT close, stated plainly rather than papered over. The step-5 verify command is an operator-supplied command line executed by `bash -c` on the DEFAULT, FLAGLESS path, against a working tree checked out from an upstream moments earlier, and it inherits this host's live push credentials. Measured on this host: `SSH_AUTH_SOCK` live with two loaded keys, `~/.ssh/id_ed25519` readable with an empty passphrase, `gh` and `glab` configs readable, 25 own-org submodule upstreams plus the parent's two mirrors in reach, with no timeout and no resource bound. Scrubbing the environment does not close it: OpenSSH resolves ~ from the passwd entry, not from `$HOME`, so a push succeeded from a fully empty environment with `HOME` redirected to

an empty directory. Closing it requires running the verify invocation inside a credential-masked sandbox (`bwrap` is present on this host and needs no privilege escalation, §6.U). **That work is owed and is not done here.** The script's header previously claimed "nothing leaving this machine" for the clean-tree path; that sentence was false and has been corrected.

A path-level governance refusal is exit 0, on purpose

`REFUSED_GOVERNANCE_DENY` reaches the summary through two different paths, and they exit differently. That asymmetry is deliberate and is the reason exit 0 is correct for the first of them:

Shape	Counter	Exit
The submodule's **path** carries no standing authorization (it is absent from `GOVERNANCE_ALLOW`) — nothing is attempted, its upstream is never contacted	`GOVERNANCE_REFUSED`	**0**
An **authorized path** whose upstream names a repository nobody authorized; an upstream identity that cannot be read; a certification that examined **zero** identities	`FAILURES`	**1**

With `GOVERNANCE_ALLOW` naming one submodule out of twenty-five, a path-level refusal is the designed steady state of **every single run**. An exit code that reports failure on every run is an exit code its operator learns to ignore — which is precisely the "route around the guard" defect this file polices elsewhere. Nothing is attempted for such a submodule, nothing is left inconsistent, and a record naming the refusal exists at rest. The *dangerous* half of governance — an authorization spent on a repository nobody ruled on — is **not** exit 0.

The residual concern is real and is not solved by changing this code: a repository in which *every* submodule is governance-refused reports exit 0 having advanced nothing. That is indistinguishable **by exit code alone** from "everything was already current". It is distinguishable from the summary (the governance bucket is reported separately and the run asserts its buckets account for every submodule it examined) and from the records at rest. Merging the two governance shapes into exit 1 would lose information rather than add it, and `tests/pipeline/test_advance_all_submodules.sh:423` pins the exit-0 contract that a

sibling suite depends on. **Assessed 2026-08-26: exit 0 is correct here; the documentation of it was wrong and has been fixed.**

- **Issues no `git push` at all**, to any remote, on any path, with or without any flag — and creates **no commit**, in a submodule or in the parent. This used to read "never force-pushes, never rewrites history, never bypasses hooks or signing (§6.T.3)"; the stronger statement is now true because the only pushing and committing code in the file was R-005 step 6, and it is gone.
- **A rejected advance is discarded entirely, never partially applied.** Both rejection paths check the submodule back out to its prior pinned commit, so the parent repository is left exactly as it was found.
- **Never destroys local work**, and no longer needs a rescue-ref mechanism to say so: a submodule carrying uncommitted work is refused *before anything is fetched or checked out*, so its work is never moved, never committed and never at risk. The refusal names every path it found. Restoration on the other rejection paths uses a plain checkout, never a discarding one.
- **The pin it compares against is the parent's index gitlink**, never the submodule's working-tree HEAD. If the two differ (`git submodule status` shows a leading `+`) the submodule is REFUSED rather than advanced from a commit nobody chose. This is what makes the non-fast-forward guard meaningful and what stops a previous run's unrestored leftovers from reading as a clean starting point.
- **Own-org upstreams only** (condition (C)). Every configured remote is classified BEFORE the fetch, over **four** URL forms, because no one of them is where git actually goes:
  - `git remote get-url` — the fetch URL after `url.<base>.insteadOf`;
  - `remote.<n>.url` — the same, as literally configured, before that rewrite;
  - `git remote get-url --push --all` — **every** destination a push would reach, after `remote.<n>.pushurl` and `url.<base>.pushInsteadOf` (`--all`, not bare `--push`: `pushurl` is multi-valued and git pushes to every value, while bare `--push` prints only the first);
  - `remote.<n>.pushurl` — the push destinations as literally configured.

  A remote on a host other than GitHub/GitLab, or under an org other than `vasic-digital/HelixDevelopment`, is refused on **any** of those forms, not skipped: a silently skipped mirror is a §6.C divergence. The push forms were absent until the round-3 review, and their absence was not theoretical — `constitution/origin` in this repository fetches from `github.com/`

`HelixDevelopment` and **pushes to `gitflic.ru`**, a provider §6.W names explicitly forbidden. (That submodule is governance-denied, so it never reached this check; the configuration *shape* is what matters.)

The push-time re-certification that used to accompany this scan is gone with the push it guarded. The **pre-fetch** scan is now the whole of condition (C), and it is still load-bearing: it gates the FETCH, which is the remaining way an out-of-scope repository's objects could end up under the pin this run stages for every clone.

- **Nothing reaches an upstream at all.** This bullet used to describe how the operator's pre-verify content was protected from being replaced by the build before being published. Nothing is published, so the question does not arise. The paths carrying the submodule's own local work are still captured — but only so the refusal can **name** them for the operator.

- **A refusal offers no consolation it cannot deliver.** It used to write the operator's pre-verify bytes into the submodule's object database so the printed sha was a real recovery handle, because the build had already overwritten the file on disk. The build no longer runs on that path, so the bytes are not merely recoverable, they are untouched — and the refusal correspondingly claims no rescue, no preservation and no reconciliation, because none was needed.

- **A gate may not pass having examined nothing.** The condition-(C) scan asserts it read at least one remote before it is allowed to certify one; `git remote` exiting 0 with empty output is "the upstream set was not learned", never "no out-of-scope upstreams found".

- **Authorization is checked against the upstream, not the directory name.** `GOVERNANCE_ALLOW` names a *repository*. Matching only the submodule path's final component let an authorization travel with a directory label, so a submodule at an allowed-looking path pointing at a different repository was advanced. The path name is still necessary; every own-org remote's repository name must now also be allowed.

- **A gate that reads zero items has certified nothing.** The zero-iteration floor that the push loop carried now lives only where it still applies: the condition-(C) scan and the governance identity check both refuse rather than certify a set they never read.

- **No privilege escalation** — plain git as the calling user (§6.U).

Hermetic test

`tests/pipeline/test_advance_all_submodules.sh` exercises all four R-005 branches against **disposable git fixtures** built under `mktemp -d` — a throwaway parent repo, a throwaway submodule, and a local bare repo standing in for that submodule's upstream. It never touches this repository's real `submodules/` tree and never reaches any network host.

Case	Fixture setup	Asserted
1	Pin already at upstream HEAD	`NO_NEWER_COMMIT`, `old_commit == new_commit`, pin untouched, exit 0
2	Upstream one commit ahead, verify passes	`ADVANCED`, submodule HEAD **and** parent index gitlink both moved to upstream HEAD, exit 0
3	Upstream one commit ahead, verify fails	`REJECTED_BREAKING_CHANGE`, submodule HEAD restored, parent pin unchanged, exit 1
4	Clean submodule; the verify hook lands a concurrent commit on the fixture upstream **during** the rebuild-and-test window	`ADVANCED` at the commit **step 2 measured**, not the concurrent one; upstream tip is still the concurrent developer's commit; upstream gained no branch; **no** rescue ref exists; exit 0
4b	Submodule carries uncommitted work	`FAILED_PRECONDITION`, pin unchanged, submodule `origin/master` unmoved (proving no fetch ran), upstream unchanged, operator's bytes untouched on disk, exit 1 — **and** `--publish-local-modifications` is exit 2 naming the removal, writes zero records, and is not mistaken for a repository path

Every assertion is on **real git state** (`git ls-files -s`, `git rev-parse HEAD`, `git show <ref>:<file>`), not merely on what the JSON record claims — so a record that lies about what it did is caught by the test rather than believed.

Falsifiability rehearsal (2026-08-21, T052 GREEN step): the step-5 failure branch was mutated to stage the parent pin (`git add`) instead of calling `restore_pin`, while still writing `parent_pin_updated: false` into the record — i.e. a record that lies. Case 3 failed with:

```
FAIL: case3(breaking-change): submodule HEAD restored to the prior
pin expected '2244ef88…', got '20b78dcc…'
FAIL: case3(breaking-change): parent pin NOT updated expected
'2244ef88…', got '20b78dcc…'
```

Cases 1, 2 and 4 stayed green, so the failure was localized to exactly the mutated path. The mutation was reverted and the suite returned to 36/36 PASS.

2026-08-22 pre-T054 audit: hardening + open review items

This script is gated behind Human Checkpoint #2 (task T054) because of its blast radius. Ahead of that review it was audited against disposable git fixtures (bare local repos standing in for upstreams; no network, no real submodule touched). Twelve failure modes were demonstrated and fixed; every one has a case in `tests/pipeline/test_advance_all_submodules_hardening.sh`.

What was fixed

#	Failure mode	What it did
1	**Uninitialized submodule → the script operated on the PARENT repo**	`git -C <empty-dir> rev-parse --git-dir` succeeds by walking UP, so the guard never fired. The parent repository was detached from its branch and its tree replaced with its own `origin/HEAD`; the run reported `ADVANCED` / `parent_pin_updated: true` / exit 0. Now guarded by `is_own_repo_root`, which requires the directory to be the root of its own work tree.
2	**A failing `git submodule status` read as "nothing to do"**	Its stderr was discarded and zero parsed rows produced `no submodules found — nothing to do` + **exit 0**. Its exit status is now checked before its output is parsed; a failure is exit 2 with git's own diagnostic.
3	**The allow-list widened runs instead of narrowing them**	`grep -qw` treats the path as a word-boundary regex, so `LAVA_ADVANCE_SUBMODULES=submodules/auth-extra` also selected `submodules/auth`. Now an exact whole-token comparison.
4		"Remote HEAD differs from the pin" is not "remote HEAD is

#	Failure mode	What it did
		newer". This project pins submodules to side branches on purpose (root `CLAUDE.md` records Containers pinned on `lava-pin/2026-05-07-pkg-vm`), and moving such a pin to `origin/HEAD` drops the pinned commit. Now refused via `merge-base --is-ancestor`, before anything is checked out.
5	**`local_modifications_pushed: true` when nothing was committed or pushed**	`pushed` was set from having *entered* step 6. The step-4 checkout can legitimately leave `git add -A` with nothing to stage (e.g. the new upstream commit adds a `.gitignore` rule covering the operator's untracked file). Now derived from a commit actually being created.
6	**`local_modifications_pushed: false` when one mirror already had the work**	On a multi-mirror push where mirror 1 accepted and mirror 2 refused, the record stated the opposite of the upstream's observable state — and force-push is forbidden, so it cannot be taken back. Now reports `true` and prints an explicit MIRROR DIVERGENCE warning.
7	**Pushed one remote's default branch name at every remote**	The branch was resolved once, from the preferred remote. A mirror whose default is `main` got a stray `master` branch while its real default received nothing. Now resolved per remote.
8	**`(cannot fast-forward)` asserted for every push failure**	Branch protection, a server-side hook, an auth failure and an unreachable host all reported as a non-fast-forward while git's real message was discarded. git's own message is now quoted verbatim.
9	**Parent-staging failure after a successful push lost the commit locally**	HEAD was moved back off a commit that was already published, with no rescue ref — reflog-only. A rescue ref is now created on that path too.
10	**A governance-deny record-write failure was swallowed by `	
11		

#	Failure mode	What it did
	A KILLED verify command was indistinguishable from a breaking change	An OOM kill or `timeout` produced a byte-identical record to a genuine test failure. The exit status (and the signal, for ≥128) is now stated in the run log.
12	**A failed restore-to-prior-pin was a lone stderr line**	The submodule is then left on the *rejected* commit and the parent tree is dirty; in a 25-submodule run that line scrolls away. Now counted and named in the run summary (`N not restored to the prior pin`).

Open items for the T054 reviewer — NOT fixed here

1. ~~**The record schema's `outcome` enum cannot express several real refusals.**~~ **CLOSED 2026-08-26 (T054).** The reviewer decision was taken: the enum grew rather than the truth shrinking. `submodule-advance-record.schema.json` gained `FAILED_PRECONDITION`, `REFUSED_NOT_FAST_FORWARD`, `REFUSED_FOREIGN_UPSTREAM`, `REJECTED_VERIFY_MUTATED_LOCAL_WORK` and `REJECTED_PARENT_STAGING_FAILED`, and every one of the eight previously record-less terminal paths now writes one — root `CLAUDE.md` condition (E). The step-7 staging-failure path no longer records `REJECTED_BREAKING_CHANGE` with a printed apology; it records its own cause. The enum widening is backward compatible: every record written before this change still validates. One case remains unrecordable and is documented as such above (an unreadable parent gitlink leaves no sha to record).

2. ~~**The governance deny-list holds only `constitution`.**~~ **CLOSED 2026-08-26 (LVA-138, operator decision).** The reviewer decision recorded here was taken in the direction this item recommended: the deny-list was inverted into a **default-DENY allow-list**, `GOVERNANCE_ALLOW`, whose sole member is `helixqa`. That is the one submodule carrying a standing authorization to track upstream unattended (root `CLAUDE.md`'s operator-authorized pin-policy waiver, Phase 4-C-1 decision Q9). Everything else — `constitution`, the sixteen pins-frozen-by-default submodules, and the seven named by no operator decision at all (`panoptic`, `superspec`, `doc_processor`, `llm_orchestrator`, `llm_provider`, `llms_verifier`, `vision_engine`) — is refused without being examined, however green its rebuild-and-test would have been.

   A default-deny list can only ever be as correct as it is complete, and the old one was 1/25 complete; a default-DENY posture is correct by construction instead, because a submodule nobody

has ruled on fails closed rather than open. The boundary is not widenable by `LAVA_ADVANCE_SUBMODULES` (that variable can only narrow a run), it is a plain array assignment so an exported environment variable of the same name is overwritten, and there is no test-only override — the suites exercise the allowed path by naming their fixture submodule `helixqa`. The T054 review attempted three bypasses (exported `GOVERNANCE_ALLOW`, a widening `LAVA_ADVANCE_SUBMODULES`, and a `BASH_ENV` file redefining `_is_governance_denied`); all three were refused.

Adding an entry to `GOVERNANCE_ALLOW` is a **governance act**: it asserts that an operator has authorized unattended advancement of that submodule, and it MUST NOT be done to make a test or a pipeline run go green.

3. **Step 6 still publishes the operator's pre-existing untracked files to a real upstream.** `has_local_mods` comes from `git status --porcelain`, which reports untracked files. Any untracked, non-ignored file a submodule already carried — a scratch note, a file a `.gitignore` does not yet cover — is committed and published to that submodule's own upstream under the message `chore: local modifications carried forward by advance-all-submodules`. This is R-005 step 6 as specified, but its interaction with §6.H (credential inviolability) deserves an explicit reviewer decision: an allow-list of paths, `git add -u` (tracked files only), or an operator confirmation gate.

   Narrowed on 2026-08-23, not closed. What step 6 publishes is now bounded by the paths the submodule carried *before* the advance; it no longer also sweeps in whatever the step-5 rebuild wrote. See finding B2 below. The residual risk is the operator's own pre-existing untracked files, which is the reviewer decision above.

4. **Never yet run against a real submodule upstream.** Every finding above came from disposable local fixtures. The fixtures cannot reproduce real-upstream behaviour such as authentication prompts, server-side hooks on GitHub/GitLab, or `ls-remote` against a host that redirects.

5. ~~**A record-write failure on one path is still swallowed by `|| true`.**~~ **CLOSED 2026-08-26 (T054).** The `cat-file -e` branch (the fetched object is absent after a successful fetch) now routes through the shared `refuse_and_record` helper, so a failed write calls `note_record_failure` like every other site, and its outcome is `FAILED_PRECONDITION` rather than a `REJECTED_BREAKING_CHANGE` asserting a breaking change nobody observed. The branch remains defensive-only — still no fixture reaches it — but it is no longer the one site that could fail silently.

2026-08-23 pre-T054 blast-radius audit

A second fixture-only audit, focused on the questions a T054 reviewer has to answer about blast radius. Three failure modes were demonstrated and fixed; each has a case in `tests/pipeline/test_advance_all_submodules_blast_radius.sh`, and each case is paired with a positive case so a blanket fail-everything change cannot satisfy the suite.

#	Failure mode	What it did
B1	**An existing-but-unwritable record directory was not a configuration error**	`mkdir -p` returns 0 for a directory that already exists whatever its mode, so its success stood in for "records can be written here". The exit-code table has always promised exit 2 and "*Nothing was attempted*" for an unwritable record directory; in fact the run fetched, checked out, verified, published the submodule's local work and staged the pin, then wrote **zero records** and reported `1 advanced` with exit 1. The directory is now probed for real before anything is attempted.
B2	**Step 6 published files the step-5 verify build had just created**	`has_local_mods` is sampled before the advance (correct), but `git add -A` ran *after* the verify command, so one pre-existing stray file armed step 6 and every artefact the rebuild wrote rode along into the commit pushed to the submodule's own upstream default branch. Demonstrated: a fixture whose only real local change was `NOTES.txt` published `build/outputs/app.apk` and `build/local.properties` alongside it. The default verify command is a whole-repository build, so that set is unbounded, and history-overwriting pushes are forbidden (§6.T.3) so it cannot be taken back. Step 6 now stages exactly the paths captured before the advance.
B3	**A failed step-6 stage was reported as `ADVANCED` with exit 0**	A failed `git add` printed a note and fell through; the following `git diff --cached --quiet` then found an empty index, the run said "*nothing left to commit*", advanced the pin, staged it and exited 0. "*Step 6 had nothing to do*" and "*step 6 could not run*" produced a byte-identical record and an identical green exit, while the operator's uncommitted work was

#	Failure mode	What it did
		left stranded in a submodule whose pin had moved. A staging failure is now `REJECTED_PUSH_CONFLICT` with the advance discarded and the pin left alone.

Two supporting changes came with B2: `git status` is now read with `-z -uall` (a path may contain a newline; an untracked *directory* must be expanded to the files it held at that moment, or anything the build later wrote into it would be re-admitted), and its **exit status is checked** — an unreadable index must not read as "this submodule carries no local work". Paths the new upstream commit has since gitignored are filtered out before staging, because `git add -- <ignored>` exits non-zero while still staging its other arguments, which would otherwise turn the legitimate "nothing left to commit" case (hardening finding #5) into a false failure. Case B2i guards that.

Checked and found clean in the same audit

- **No history-overwriting or hook-bypassing flag anywhere.** All ten mutating git calls were enumerated; the push refspec is `HEAD:refs/heads/<branch>` with no leading `+`. Exactly one write touches the parent repository, `git add -- <sub-path>`; there is no commit, push, checkout, reset, clean, stash or tag in it (§6.T.3).
- **A fetch failure is distinguishable from "no newer commit".** An unreachable upstream and a submodule with no remote are both counted failures with git's own diagnostic, no record, exit 1, and the pin untouched.
- **A killed verify leaves the pin put.** `kill -KILL` and a genuine non-zero verify both restore the prior pin and exit 1; the run log distinguishes them (`exit status 137 — KILLED by signal 9`), the record at rest does not — that is open item 1.
- **A symlinked submodule directory is refused.** `git submodule status` itself exits 128 ("*expected submodule path … not to be a symbolic link*"), which the hardening fix #2 turns into exit 2 with nothing attempted, even when the symlink points at the parent repository.
- **No record-filename collisions** across the twenty-five real submodule paths under the sanitizer.

2026-08-26 T054 Human Checkpoint #2 review

The review verdict was **APPROVE WITH FIXES — not safe to point at a real submodule upstream until BLOCKER-1/2/3 are closed**. All three are closed here, along with four of the six

should-fix items and one nit; each was reproduced against a disposable fixture *before* the fix, and each has a red-before-green case in `tests/pipeline/test_advance_all_submodules_t054.sh`.

Blockers

#	Failure mode	What it did, and what changed
B1	**The pin was read from the submodule's HEAD, not the parent's index**	`old_commit` came from `git -C <sub> rev-parse HEAD`. The pin is the parent index gitlink; the two differ whenever `git submodule status` shows a leading `+`. Every downstream decision — the fast-forward guard, the record's `old_commit`, `restore_pin`'s target — was therefore evaluated against the wrong reference. Demonstrated: with the parent deliberately pinning side-branch commit `S` and the tree left at its ancestor `A`, `merge-base --is-ancestor A B` passed, the pin moved to `B`, and `S` became unreachable from the staged pin — exactly the `lava-pin/2026-05-07-pkg-vm` case the guard's own comment says it exists to prevent. The record named `A` as `old_commit`, so the evidence at rest misstated what was advanced FROM. The pin is now read once from `git ls-files -s` (the idiom the governance branch already used, for the reason its comment already gave), and a working tree that is not sitting on the pin is refused outright.
B1b	**An unrestored run's leftovers read as a clean run**	Same root cause. When the step-5 rebuild dirties a tracked file that differs between the two commits, the non-discarding restore checkout is legitimately refused and the submodule is left parked on the REJECTED commit. The next run read that commit as the pin and reported `NO_NEWER_COMMIT`, **exit 0**, and a green record — for a submodule whose pin had never moved. Reading the pin from the index closes it, and the HEAD-vs-pin guard turns the leftover state

#	Failure mode	What it did, and what changed
		into an explicit refusal naming the restore command.
B2	**The staged path SET was frozen; its CONTENT was not**	The 2026-08-23 fix (B2 above) captured the local-modification path list *before* the advance, but `git add` still runs *after* the step-5 verify command. So for any path the operator had **already** modified and the build **also** touches, the build's bytes were committed and pushed to the submodule's own upstream default branch. Both directions were demonstrated: an operator's hand-written file **overwritten** by build output and published, and a tracked file that existed on `origin/main` **deleted** from it by a build's clean task. Force-push is forbidden (§6.T.3), so neither can be taken back. Those paths are now content-fingerprinted before the advance and re-checked before staging.
B3	**`LAVA_PIPELINE_RUN_ID` was interpolated into a string fed to `bash -c`**	The default verify command was built by splicing the run id between single quotes. A `'` in the value closed the quoting and the remainder was parsed as shell. Demonstrated with `LAVA_PIPELINE_RUN_ID="run'; echo … > MARK; exit 0; '"`: arbitrary commands ran, **and** the injected `exit 0` made the step-5 gate pass unconditionally — the pin advanced with **zero verification** and was recorded `ADVANCED`, while the build script it claimed to have run did not even exist. A control run with a benign run id correctly rejected. This is a security defect and an anti-bluff defect at once: the gate reported a verification it never performed, defeating research.md R-005's implementation note 1. The run id is now validated as a plain token and passed to the verify command **positionally as `$3`**; it never becomes syntax. The same validation closes the `RECORD_DIR` path-traversal nit.

Why B2 refuses rather than reverts. A silent revert would discard whichever side is not chosen — real work either way, the operator's or the build's — without anyone being told, which is the same class of quiet data loss the rest of this script exists to avoid. Refusing names the exact paths, leaves both versions on disk, discards the advance and restores the pin, so a human decides. Isolating the verify step instead (running it against a throwaway copy) was considered and rejected: the default verify command is a whole-repository build that must run against the advanced tree in place, so a copy would not be verifying the thing being advanced.

Whole-file injection audit. Every `bash -c`, `sh -c`, `eval` and string-built command in the script was enumerated: **one** `bash -c` site (the step-5 verify invocation) and **one** command string built from an untrusted value (the default `VERIFY_CMD`, which embedded `RUN_ID`). **Zero** `eval`, **zero** `sh -c`, **zero** `xargs`. Both sites are fixed. The record writer was already safe by construction — `jq -n --arg/--argjson`, or a `<<'PYEOF'`-quoted python heredoc taking values through `argv`.

Should-fix items

Item	What changed
Condition (C) was entirely absent	Root `CLAUDE.md`'s Automated Pipeline Pin-Advance Path requires the path to advance only submodules whose configured upstreams are `vasic-digital/*` or `HelixDevelopment/*` on GitHub or GitLab. Nothing anywhere read a remote URL. Demonstrated: a submodule carrying a second, unauthorised remote had the operator's work **pushed to it**, and every foreign remote was **fetched from** before anything else happened. Now every configured remote is classified before the fetch and an out-of-scope one is refused with a `REFUSED_FOREIGN_UPSTREAM` record.
Eight refusal paths wrote no record (condition (E))	See open item 1 above — closed by widening the enum and routing every terminal path through `refuse_and_record`.
A zero-iteration push loop read as success	`while … done < <(git remote)` hides the producer's exit status. With `git remote` failing transiently (fault-injected at the git boundary by a PATH shim; the script itself was not modified), the loop body never ran, no failure was recorded, and the parent pin was

Item	What changed
	staged at a commit **no remote had** — every other clone would then get `fatal: reference is not a tree` from `git submodule update`. The remote list is now captured into an array through a file with its exit status checked, and a created commit that no remote accepted is a `REJECTED_PUSH_CONFLICT` with the work preserved under a rescue ref.
An `LAVA_ADVANCE_SUBMODULES` typo examined nothing and exited 0	A token naming no submodule selected NOTHING; the run reported `0 advanced, 0 already current, 0 rejected/failed`, wrote zero records, and exited 0 — indistinguishable from a clean run. That is the vacuous-pass family this repository has around fifty recorded instances of. Unmatched tokens are now exit 2 with the present submodule set printed. The summary also gained a separate `N submodule(s) examined` line, and a run that examined zero submodules is refused outright. The token list is additionally split with `read -ra` so it can no longer glob-expand against the caller's working directory.
The governance refusal's default reason made a specific claim	It asserted a fact about the absence of operator decisions *anywhere on record* — a claim that goes false the moment an operator authorizes a submodule without that table being updated, which is precisely the confidently-wrong drift its own comment says the default exists to avoid. It was already at its edge: of the seven submodules that arm fires for, `panoptic` does appear once in root `CLAUDE.md`, inside a §6.L historical narrative rather than a pin-policy decision. It now states only what is true by construction — the submodule is absent from `GOVERNANCE_ALLOW` — while still pointing at where an authorization would have to be recorded. The other two arms were verified byte-accurate against root `CLAUDE.md` and are unchanged.

Item	What changed
`ADVANCED_COUNT` counted a submodule whose record failed to write (nit)	A single submodule was reported as both `1 advanced` and `1 rejected/failed`. The counter now moves only when the record really landed.
`git submodule status 2>&1` merged stderr into the parsed stream (nit)	A diagnostic emitted on a *successful* run could be parsed as a submodule path. The two streams are now captured separately.

Disclosed residual gaps — real, and NOT closed here

1. **Condition (C) does not classify filesystem-path remotes.** A remote URL that names no network host (`/path/to/repo.git`, `./relative`, `file://…`) is classified `LOCAL_PATH` and permitted. The reasoning: there is no third party to receive an unattended publish, and it is the only remote shape a hermetic fixture can have — a suite that cannot reach the push path cannot prove the push path is safe. `insteadOf` URL masking was tried as a way to give fixtures own-org-shaped URLs and make the check strict; it does not work, because `git remote get-url` returns the **rewritten** URL. So a local scratch mirror configured inside a submodule clone would still be pushed to. No real Lava submodule has such a remote today, and the condition-(C) check would catch any network-hosted one.

2. **Never yet run against a real submodule upstream.** Unchanged from the 2026-08-22 audit. Every finding in this document came from disposable local fixtures, which cannot reproduce authentication prompts, server-side hooks on GitHub/GitLab, or `ls-remote` against a host that redirects.

3. **Step 6 still publishes the operator's pre-existing untracked files.** See open item 3 above — narrowed twice (path set, then content) but the underlying reviewer decision about untracked files and §6.H is still open.

Condition (C) measured against this repository (read-only, 2026-08-26)

Classifying every configured remote of all 25 registered submodules with the script's own `_remote_url_class`, without contacting anything:

- **23 of 25 submodules are entirely `OWN_ORG`**, including `submodules/helixqa` — the only submodule `GOVERNANCE_ALLOW` permits — so the new scope check does not block the one advance this pipeline may actually perform today.

- **`superspec`'s `origin` classifies FOREIGN**: it is on GitHub but under an org that is neither `vasic-digital` nor `HelixDevelopment`.
- **`constitution` carries four FOREIGN remotes**: two on GitFlic, one on GitVerse, and a GitLab remote under a foreign org. Three of those are the §6.AD.1 carve-out — HelixConstitution is HelixDevelopment-owned and ships its own 4-upstream `install_upstreams.sh`, which §6.W scopes to *within* that submodule's git-dir rather than to the parent.

Neither submodule ever reaches the condition-(C) check: both are refused by the governance allow-list first, which runs before the fetch, before the scope check, and before anything else. Condition (B) — `constitution` excluded by a default deny that requires a code change to override — is satisfied by construction.

Also confirmed read-only: all 25 submodules currently show a leading space in `git submodule status`, i.e. HEAD equals the parent index gitlink for every one. The new HEAD-vs-pin refusal therefore fires on none of them today. That is what made BLOCKER-1 *latent* rather than active: a single `git checkout` inside any submodule, or one unrestored run, arms it.

Falsifiability rehearsals (§6.J, 2026-08-26)

Each fix was reverted **alone**, the suite re-run, the verbatim failure captured, the fix restored, and the script proven byte-identical to its pre-mutation state by sha256. Every rehearsal ended with the suite back at 0 failures.

Fix reverted	First failure, verbatim	Failing assertions
B1 — pin from the parent index	`FAIL: T1: exit code (0 would certify a dropped pin) expected to differ from '0' but was identical`	10
B2 — verify-step content check	`FAIL: T2a: exit code (0 would certify a publish of the build's bytes) expected to differ from '0' but was identical`	10
B3 — run-id validation + positional	`FAIL: T3a: exit code (2 = configuration error, nothing attempted) expected '2', got '0'`	8
Condition (C) scope check	`FAIL: T4(foreign-host): outcome names its own cause expected 'REFUSED_FOREIGN_UPSTREAM', got 'FAILED_PRECONDITION'`	3
Zero-iteration push loop	`FAIL: T5: exit code (0 would certify an unpublished pin) expected to differ from '0' but was identical`	5

Fix reverted	First failure, verbatim	Failing assertions
Allow-list token validation	`FAIL: T6: run output does not contain 'no submodule for'`	1
Governance refusal default arm	`FAIL: case6(panoptic): refusal did not state the reason for its class.`	2

Two of these deserve a note rather than a bare number. Reverting the allow-list token validation produced only **one** failing assertion because the run then falls through to the independent `EXAMINED_COUNT == 0` guard, which also exits 2 — the two guards are genuine defence in depth, and the message assertion is what discriminates between them. Reverting the condition-(C) check produced three, all on the outcome value, because the fetch to the unreachable foreign remote then fails anyway; the exit code and the untouched pin are the *same* either way, so only the recorded cause tells the two apart.

2026-08-26 T054 re-review — the second round

The re-review re-proved all three blockers closed from its own fixtures and returned **APPROVE WITH FIXES**, with five further items. All five are closed here, each reproduced against a disposable fixture first and each covered by a regression case in `tests/pipeline/test_advance_all_submodules_t054.sh` (cases T8–T16) that goes red when its fix alone is reverted.

Item	Was	Now
The safe-operating precondition	Unchecked and unstated. A stray `.notes.swp` was committed and published to another repository's default branch, exit 0, `ADVANCED`.	Checked and stated. Unclean tree ⇒ `FAILED_PRECONDITION` unless `--publish-local-modifications`.
Mode-only change by the build	`chmod 755` on an operator's untracked file published `100755` upstream, exit 0, `ADVANCED`.	Refused. The comparison is a `write-tree` over what `git add` would record, and the per-path description carries the mode.
Nested submodule gitlink	Any directory answered `directory`, so a build moving a nested submodule's	Refused. A directory that is its own work-tree root is described by that tree's HEAD, and the

Item	Was	Now
	HEAD published the build's gitlink. `submodules/helixqa` declares **27** nested submodules.	`write-tree` comparison covers gitlinks by construction.
Condition-(C) scope gate	Passed having examined **zero** remotes (proven with a `PATH` shim); the run then fetched, advanced and staged.	Asserts at least one remote was read, mirroring the guard the push loop already had.
Refusal honesty	Claimed "both versions are left on disk" and printed a `blob:<sha>` that named an object stored nowhere.	The blob is written at capture time, so the sha resolves; the message distinguishes what is recoverable from what is not.
`LOCAL_PATH` remotes	Permitted in production. An unauthorised same-filesystem repository received the operator's work, exit 0, `ADVANCED`.	Refused unless `--allow-local-path-remotes`. Measured: **zero** `LOCAL_PATH` remotes exist across all 25 submodules, so closing it costs production nothing.
Run id `.` / `..`	Accepted; relocated the run's records out of its run directory.	Exit 2, nothing attempted.
Submodule staging failure	Recorded `REJECTED_PUSH_CONFLICT` — a push that is never attempted on that path.	Records `REJECTED_SUBMODULE_STAGING_FAILED` (new enum value).
Allow-list keyed on path basename	`vendor/x/helixqa` pointing anywhere was advanced.	Path basename **and** every own-org upstream's repository name must be allowed — conjunctive, not a replacement.

Two honest residuals

1. **The recovery handle has two providers.** Reverting the `hash-object -w` alone leaves the suite green, because `_captured_tree`'s own `git add` writes the same blob into the same object database as a side effect (measured: a blob absent from the object DB before a run is present after a bare `_captured_tree` call). The `-w` stays because it is the *explicit* provider — a guarantee that depends on another function's side effect is a coincidence that has not broken yet, not a guarantee. The

message half of the fix is singly discriminated and does go red on its own.
2. **The content guard is deliberately redundant.** Either the extended per-path description *or* the `write-tree` comparison catches the mode and gitlink cases alone; both had to be reverted together to turn T9/T10 red. That is defence in depth rather than a gap, and it is recorded here so a future reader does not mistake the surviving single-mutation for a missing test.

2026-08-26 T054 round-3 review — the third round

The round-3 review re-proved all six round-2 fixes from its own fixtures and returned **APPROVE WITH FIXES**, with six more items. All six are closed here, plus **four further escapes of the same class** that the hunt for a fourth turned up. Every one was reproduced against a disposable `mktemp -d` fixture BEFORE its fix, and every one has a regression case in `tests/pipeline/test_advance_all_submodules_t054.sh` (T17–T26) that goes red when its fix alone is reverted.

Item	Was	Now
R3-F2 — condition (C) classified the FETCH url	`remote.origin.pushurl` sent the operator's work to a repository the scope check never examined, exit 0, `ADVANCED`; and a `pushInsteadOf` rewrite made the run genuinely attempt to contact a host the classifier calls FOREIGN.	Four URL forms classified, push destinations included, and the certification re-taken immediately before each push.
R3-F1 — the commit was not scoped (3rd escape)	A verify command that ran `git add build-leak.txt` published it: `git commit` takes the whole index, not the staged paths.	The index must equal `HEAD` before staging, the commit is `--only -- <audited paths>`, and the finished commit is compared against the fetched

Item	Was	Now
		upstream commit.
ESC-4 — `HEAD` moved by the build (4th escape)	A verify command that COMMITTED its own output published that commit as the carry-forward commit's parent. Scoping the commit does not help — the content is in the parent, hence in the tree.	`HEAD` must still equal the fetched commit before anything is staged.
ESC-5 — a remote added AFTER the scan (5th escape)	`git remote add rogue <repo>` inside the verify command; the never-examined repository received the work, exit 0, `ADVANCED`.	Push destinations are re-classified at push time, with a zero-destination floor.
ESC-6 — a pre-commit hook inside the submodule (6th escape)	Measured: under `git commit --only` git hands the hook a temporary index, and a hook running `git add` has its file land in the commit git builds.	The finished commit's diff against the fetched upstream commit must lie entirely inside the audited path set.
ESC-7 — `push.followTags` (7th escape)	A build-created annotated tag reached the upstream, exit 0, `ADVANCED`. A tag is a ref another repository now carries, and §6.T.3 forbids taking it back.	`--no-follow-tags --recurse-submodules=no` on every push.
R3-F3 — the argv-only property was asserted, not checked	`BASH_ENV` carrying `declare -r PUBLISH_LOCAL_MODS=true` made the script's own reset fail silently under `set +e`; the run published, exited 0, and **printed "--publish-local-modifications was given" when argv gave no such flag**.	The reset's exit status is checked and the final values are compared against argv. Either disagreeing is exit 2, nothing attempted.
R3-F4 — the identity gate	Zero iterations read as "every upstream names an authorized repository"; an unparseable own-	Floored, with `LOCAL_PATH`

Item	Was	Now
had no zero-item floor	org name was skipped rather than refused.	remotes counted explicitly so a nameless certification is **stated** rather than silent; an unreadable name refuses.
R3-F5 — the scan's comment overclaimed	It called the effective fetch URL "what git will actually contact", which the `pushurl` proof contradicts.	The comment now enumerates all four forms and says what each one is.
R3-F6 — attribute divergence reads as a mutation	An upstream commit introducing `text=auto` changes the staged tree id for byte-identical content, producing a `REJECTED_VERIFY_MUTATED_LOCAL_WORK` refusal with nothing to say the cause might not be a mutation.	The refusal is kept (it is the safe direction) and now names attribute divergence as a candidate **only when the two commits genuinely differ in an attribute file**, so it cannot become a routine excuse.

The round-2 residual, closed

The round-2 report disclosed that reverting `hash-object -w` alone left the suite green, because `_captured_tree`'s own `git add` writes the same blob as a side effect. The round-3 review confirmed that diagnosis holds **but does not generalize**: under a text attribute the two providers write *different* objects — `--no-filters` stores the raw bytes, `git add` the normalized ones — and the sha the refusal

prints is the raw one, so `-w` is then the sole provider. T12's fixture now carries `*.txt text=auto eol=lf` and a CRLF file, and asserts the two blob ids differ before asserting the handle resolves. Reverting `-w` alone now fails with `the printed sha IS a recovery handle for the operator's bytes expected 'OPERATOR SECRET NOTE', got '<NOT IN THE OBJECT DATABASE>'`.

Honest residuals from this round

1. **A refusal after the commit can leave the submodule unrestored.** When a pre-commit hook has staged a file, `restore_pin`'s `git checkout --detach` is refused (`untracked working tree files would be removed`), so the submodule stays on the rejected commit. Nothing is published and the run says so loudly — `THESE SUBMODULES ARE LEFT ON A REJECTED COMMIT` — and the discarded commit is preserved under `refs/lava-advance-rescue/`. It is not forced: `checkout -f` is the flag family this script refuses on principle.
2. **`git commit --only` is not sufficient by itself, and is not claimed to be.** Measured on a fixture, a pre-commit hook's `git add` lands in the commit git builds under `--only`. What closes that is the comparison of the finished commit against the fetched upstream commit — a guard written against the published artefact rather than against an enumeration of the ways in, which is deliberate after three consecutive reviews each found one more way in.
3. **`<own-org>/<anything>/<allowed-name>` still matches the identity check**, which compares the URL's last path segment. Unchanged from round 2, where it was recorded as needing a reviewed `.gitmodules` edit to exploit.

2026-08-26 T054 round-4 review — the fourth round, and the end of the enumeration game

The round-4 review re-derived every round-3 conclusion from its own fixtures, verified all eleven round-3 fix items, and returned **APPROVE WITH FIXES** with ten more — including an **eighth** and a **ninth** publish escape. Fixing those turned up a **tenth**. All are closed here, each reproduced against a disposable `mktemp -d` fixture BEFORE its fix, each with a regression case (T27–T39) that goes red when its fix alone is reverted.

The structural finding, and what changed because of it

The reviewer's diagnosis is the most useful sentence in four rounds of review:

> the post-commit invariant is a TREE diff, read through the repository the adversary controls, while `git push` transmits HISTORY from the true objects.

Round 3 had already tried to stop enumerating routes by measuring the published artefact. It was still defeated twice, because the *measurement* was the weak part, not the *strategy*. What is measured now is what a transfer actually carries, read through a view the submodule cannot rewrite:

	question	instrument
PARENTAGE	is this one commit on the fetched upstream commit?	`rev-list --parents -n1` must equal `<final> <new>`
HISTORY	which COMMITS would the transfer carry?	`rev-list <final> --not <new>` must be exactly `<final>`
OBJECTS	which OBJECTS would the transfer carry?	`rev-list --objects <final> --not <new>`; every path-bearing entry must be an audited path, something under one, or a directory on the way to one
TREE	which PATHS differ?	`diff --name-only` — kept because object enumeration has its own blind spot: a blob whose content already exists upstream is not transferred, so a new path with duplicate content, and a **mode-only** change, are named by the tree and not by the object list
MESSAGE	is the published message the one this run wrote?	`log -1 --format=%B` compared against a repeated literal

Every one of those reads goes through **`gith`** — `git --no-replace-objects -c core.commitGraph=false` — which removes the two mappings the submodule's own repository can rewrite underneath a guard: `refs/replace/*` (substitutes objects on READ while the

transfer sends the real ones) and the commit-graph cache (a hand-authorable cache of the DAG that git trusts without verifying). And because "the reader ignores replace refs" is itself a claim, the script **proves it** on a throwaway repository before touching any submodule, and exits 2 if it cannot.

The push refspec's source is now the **immutable object id**, never `HEAD`.

The ten escapes, and the four items that are not escapes

Item	Was	Now
R4-F1 — 8th escape: a fabricated MERGE	A `post-commit` hook re-points HEAD at a `commit-tree` merge whose tree is **byte-identical** to the honest commit's. The tree diff saw `operator.txt`; the transfer carried a fabricated root commit and a credential-dump blob onto the upstream's default branch. `EXIT=0`, `ADVANCED`.	PARENTAGE + HISTORY: the finished commit must have exactly one parent, it must be the fetched upstream commit, and the transfer must carry exactly one commit.
R4-F2 — 9th escape: `refs/replace/`	A `post-commit` hook publishes `refs/replace/<final>` pointing at a benign commit. Every guard read the benign object; the push sent the real one, and `leak.txt` (a credential dump) landed **in the upstream default branch's TREE**. `EXIT=0`, `ADVANCED`.	Every gating read goes through `gith`, whose behaviour is self-tested before use.
TENTH escape: `HEAD` in the push refspec	The refspec was `HEAD:refs/heads/<branch>`. `HEAD` is a **mutable reference** git resolves at transfer time, after every guard finished. A `pre-push` hook fires inside the FIRST mirror's push and moves it; the second mirror received a different commit. §6.W mandates two mirrors, so this is the production shape. Observed: `EXIT=0`, `ADVANCED`, `PREPUSH-LEAK.txt`	The refspec source is `$final_commit`. The submodule's HEAD must still equal it before the parent pin is staged, and the staged gitlink is **read back** and compared afterwards.

Item	Was	Now
	on one mirror, the honest commit on the other (a §6.C divergence reported as success), and the **parent pin staged at a commit the record does not name**.	
R4-F3 — a replace ref inverts the fast-forward guard	`merge-base --is-ancestor` answers from the object graph, so a graft-style replace ref made an unrelated root report as an ancestor — dropping deliberately-pinned side-branch work, the one thing that guard exists to prevent.	Read through `gith`.
R4-F4 — the push-time re-check re-took half a certification	It called `_remote_url_class` and nothing else, so an own-org repository nobody authorized passed it and the run **genuinely attempted the push**; only the fixture's blocked transport stopped it.	The push-time re-check now re-takes IDENTITY as well as SCOPE.
R4-F5 — the commit MESSAGE was published unexamined	A `commit-msg` hook rewrote it and `chore: carried forward | LEAKED-VIA-COMMIT-MESSAGE` reached another repository's default branch. The invariant is `--name-only`.	The finished commit's `%B` must equal the literal this run passed.
R4-F6 — `GOVERNANCE_ALLOW` was the unchecked assignment	R3-F3 hardened the two safety switches on an honesty argument and left the **stronger** boundary unhardened. `BASH_ENV` widening it **advanced `constitution`** — exit 0, no diagnostic. `BASH_ENV` is set on the operator's host, and `constitution/origin` carries six push URLs, two on §6.W-forbidden providers.	The assignment's status is checked and its value compared to the literal; so are the three identity functions, on known inputs, because `readonly -f` is the same channel.
R4-F7 — a record naming a cause nobody observed	`git commit` refusing (reachable via a `.git/MERGE_HEAD` the build writes) was recorded	New enum value `REJECTED_SUBMODULE_COMMIT_FAILED`.

Item	Was	Now
	`REJECTED_PUSH_CONFLICT` with **zero pushes attempted**.	
R4-F8 — a permanent FALSE refusal	`git status --porcelain -uall` names an untracked nested git repository `nested/`; `git diff` names it `nested`. The audit compared the two unequal forms, so that shape could never be carried forward.	Both sides have any trailing slash trimmed.
R4-F9 — one destination reported twice	`_push_destinations_in_scope` lacked the de-duplication the pre-fetch scan has, so a refusal overstated how many destinations were out of scope.	De-duplicated.
R4-F10 — the anti-vacuity floor was itself vacuous	`R3_ESCAPES_EXAMINED` and `CASES_RUN` counted case **entry**. The reviewer gutted all six of T18's assertions and the suite still reported "*examined 6 routes / 48 cases / ALL CASES PASSED*", exit 0.	Every assertion is counted where it executes; each escape case must assert a real minimum inside its own window; and the suite carries a total-assertions floor. The same mutation now fails on three counts.
Residual 3 — identity on a substring	`_remote_repo_name` took the URL's last path segment, so `vasic-digital/sub/helixqa` passed the identity gate. Measured: all 25 own-org URLs in this repository are flat `<org>/<repo>`.	Exactly two components, or the name is unreadable — and unreadable refuses.

Hunted and found closed, in the same pass

Four further transmission routes were probed on disposable fixtures and none opened: `refs/notes` created by the build with a matching `remote.<n>.push` refspec (the explicit refspec wins; no note ref reached the upstream); `remote.<n>.mirror = true` (the push is refused outright — an unrelated branch and tag on the upstream survived untouched); an `objects/info/alternates` donor repository (nothing of the donor's reached the upstream, and the honest path still advanced); and a benign `commit-graph` (does not break the honest path, and is ignored where it would gate a decision).

Honest residuals from this round

1. **Exit 1 is sticky.** Assessed by the reviewer as *acceptable* — nothing is published, the pin is untouched, a rescue ref preserves the discarded work, the summary is loud, and the next run refuses rather than mistaking the leftover for a clean start. It is recorded under **Exit codes** above because the consequence belongs to the caller: T055 must page a human, never retry.
2. **A refusal after a successful publish is still a refusal.** When the carry-forward commit was audited and pushed, and only the *pin* check then fails (the tenth escape's shape), the commit is legitimately on the mirrors and the run reports `REJECTED_PARENT_STAGING_FAILED`. That is correct — the published content passed every guard; what failed was recording it — but the operator sees a failed run whose upstreams did move.
3. **The blind spots of each measurement are covered by another, not by one complete instrument.** Object enumeration cannot see a duplicate-content blob at a new path or a mode-only change; the tree diff cannot see history. Both run; neither is claimed to be sufficient alone.

2026-08-26 T054 round-5 review — the two findings outside the flag boundary

Rounds 1–4 all ended inside `--publish-local-modifications`. **Round 5 is the first whose findings fire on the DEFAULT, flagless invocation**, and neither of them publishes anything: one *writes into a repository nobody named*, the other *reaches a verdict having examined nothing*. Both were reproduced on disposable `mktemp -d` fixtures before being fixed, and each fix was reverted afterwards to confirm the new tests fail without it.

The review's third finding — the eleventh publish escape, the commit object's own header bytes — is **deliberately NOT addressed here**, because the operator is separately deciding whether to remove `--publish-local-modifications` altogether, which would moot it along with the whole publish path.

id	what it was	now
R5-F2	**The round-4 reader self-test wrote into whatever `GIT_DIR` named.** `git init --template=<empty> "$_selftest_dir"` was isolated from `init.templateDir` and `core.hooksPath` — and not from git's own environment, which decides *which*	The ambient git environment is **removed at the top of the script and the removal is PROVEN** — `unset` is an assignment, and a `declare -r` in

id	what it was	now
	repository it initialises at all. With `GIT_DIR` inherited and **no flags given**: `EXIT=0`, two commits created in a repository the caller never mentioned, its branch tip moved off the operator's work, its index left dirty with a `decoy.txt` absent from its worktree, and **a `refs/replace/*` ref installed and left behind** — the ninth escape's own instrument, planted by the guard written to close the ninth escape. Measured: `git submodule foreach` exports `GIT_DIR`; every commit-time hook exports `GIT_INDEX_FILE`.	`BASH_ENV` defeats an assignment silently under `set +e`, exactly as the switch-reset check already knows. A surviving name is exit 2 before any git command runs. The self-test additionally runs inside **one subshell** that removes them again (`git init` included, since that is the call `GIT_DIR` redirected) and asserts `git rev-parse --show-toplevel` really is its own throwaway directory.
R5-F3	**`"has no submodules — nothing to do"` exited 0 with zero records** without ever consulting `.gitmodules`. The enumeration comes from `git submodule status`, which reads the INDEX.	`.gitmodules` is now read **as a file, with `sed`, never through git** — a second, independent authority, because reading it through the same git would make the cross-check agree with itself by construction. Every declared path must appear in the enumeration or the run is exit 2 with nothing attempted.
R5-F3′ (found while fixing R5-F3)	**The zero case is the loudest projection of the defect, not the defect.** A floor that fires at exactly zero is a floor with one stair. With one gitlink of two dropped from the index — no environment variable at all, just `git rm --cached` — the	The floor is stated over the **SET**, not over its size.

id	what it was	now
	run reported `EXIT=0, 1 submodule(s) examined, 0 rejected/failed`: a clean verdict over a corpus that lost half its members, never naming the one it did not look at.	
R5-F3″ (found by attacking the fix above)	**The new floor fired correctly on a submodule path containing a space, and then misstated why.** `git submodule status` prints `<sha> <path> (<describe>)` and the enumeration splits on whitespace, so `submodules/helix qa` truncates to `submodules/helix` — a *different* path every later step would have operated on. The refusal blamed a missing gitlink for a gitlink that was present. Safe, and wrong; by this file's own standard a refusal that misstates its own cause is a small bluff.	The three causes that reach this state are now distinguished and the truncation one is named for what it is.
R5-F2′ (found by rehearsing the R5-F2 fix)	**A pre-existing false refusal on a third axis nobody had checked.** The self-test states its ident with two `git config user.*` calls, and `GIT_AUTHOR_*` / `GIT_COMMITTER_*` silently override config. Against the pristine round-4 script: `GIT_AUTHOR_DATE=not-a-date` → `EXIT=2`; `GIT_COMMITTER_NAME=` → `EXIT=2`, both with "*the hardened object reader could not be PROVEN to ignore `refs/replace/*`*". git rejects both outright, so the self-test's `git commit` failed and the run refused — the pipeline blocked on a guard that was working perfectly. Same defect class as the one the `--template=` isolation was added to avoid, in the same direction.	Those six names are removed **inside the self-test's subshell only**. They are deliberately left alone at the top of the script: they decide the ident bytes of the commit R-005 step 6 publishes, which is publish-path behaviour and the subject of the open decision above. Changing it there would be making that decision silently.

id	what it was	now
R5-F4 (the twelfth, found by looking for one)	**The summary did not account for every submodule it said it examined.** A governance-refused submodule landed in no bucket, so `0 advanced, 0 already current, 0 rejected/failed` was the summary of a run in which a submodule had been *refused* — the same text as a run in which everything was already current. And the per-submodule line said "*NOT advancing, and NOT examined*" while the summary counted that same submodule among the "*N submodule(s) examined*": two lines flatly contradicting each other.	Governance refusals are their own reported bucket, the per-submodule line says what did and did not happen, and the script asserts before exiting that the buckets account for every submodule it examined.

What was deliberately NOT changed, and why

- **The eleventh publish escape (commit-object header bytes) — RESOLVED 2026-08-26 by removal, not by a guard.** `--publish-local-modifications` and R-005 step 6 were deleted; the escape lived in the commit this script no longer makes. See "R-005 step 6 was removed" above.
- **`GIT_AUTHOR_*` / `GIT_COMMITTER_*` at the top of the script** — they used to decide the ident bytes of the step-6 commit. With no commit anywhere they decide nothing this script writes, and they are STILL left alone for a different and now the only reason: they are part of the environment the operator-supplied step-5 verify command inherits, and silently changing what a build sees is not a decision to take as a side effect. They remain neutralised inside the reader self-test's subshell, where that neutralisation is independently load-bearing (a malformed `GIT_AUTHOR_DATE` makes git reject the ident outright and used to turn the self-test into a false exit-2 refusal).
- **`GIT_CONFIG_GLOBAL` / `GIT_CONFIG_SYSTEM` / `GIT_CONFIG_NOSYSTEM`** — these *narrow* which config git reads; a caller sets them to isolate a run from `~/.gitconfig`. Removing them would re-admit that config: a weakening dressed as hardening.
- **`GIT_CEILING_DIRECTORIES` / `GIT_DISCOVERY_ACROSS_FILESYSTEM`** — these only stop repository discovery walking upwards. Removing them *widens* the walk, so a no-argument invocation could resolve an outer repository the operator meant to fence off. Constraining is not redirecting.

- **`GIT_SSH_COMMAND` / `GIT_TERMINAL_PROMPT`** — safety controls the hermetic suites rely on to guarantee no fixture can open a connection.
- **The exit code for a governance refusal.** It is `0` today and stays `0`; `tests/pipeline/test_advance_all_submodules.sh:423` asserts that explicitly. **Assessed 2026-08-26 and deliberately kept** — see "A path-level governance refusal is exit 0, on purpose" above for the asymmetry that makes it correct (the *dangerous* half of governance already exits 1) and for the residual that remains open. What WAS wrong was the script's own exit-code table, which claimed exit 0 meant "every submodule ended `ADVANCED` or `NO_NEWER_COMMIT`" — false on every run in which a path-level governance refusal occurs, which is every run. That statement is fixed; the behaviour and the sibling suite's assertion are untouched.
- **A third reading that has never been observed to fire.** The empty-corpus claim consults the parent index's gitlinks as well. On git 2.50.1 that state does not reach the check: `git submodule status` refuses it first with `fatal: no submodule mapping found in .gitmodules for path '<path>'` and exit 128, which the enumeration's existing exit-status guard turns into exit 2. The reading stays as a backstop for an enumeration that answers 0 instead of 128, and is described as one — an unreachable guard advertised as a live one is itself a small bluff.

Falsifiability rehearsals

Each fix was reverted **alone**, the suite re-run, and the script restored and verified byte-identical by sha256:

mutation	verbatim first failure
top-level environment removal neutered	`FAIL: T40/GIT_DIR: the run still examined the submodule it was given does not contain '1 submodule(s) examined'`
the self-test's own removal neutered	`FAIL: T47/GIT_AUTHOR_DATE=not-a-date: exit code (a false refusal blocks the pipeline) expected '0', got '2'`
the `.gitmodules` corpus floor neutered	`FAIL: T42: exit code (a green verdict over an unexamined corpus) expected '2', got '0'`
the truncation diagnostic neutered	`FAIL: T48: the refusal names the cause that actually occurred does not contain 'declared WITH WHITESPACE'`
the governance bucket counter neutered	`FAIL: T45: exit code must not be the corpus refusal (2) expected to differ from '2' but was identical`

Maintenance

When this script is modified, update this document in the same commit (`CM-SCRIPT-DOCS-SYNC` requires it). Per §11.4.18 the documentation MUST stay in sync with the codebase.

Cross-references

- `scripts/advance-all-submodules.sh` — the script itself
- `tests/pipeline/test_advance_all_submodules.sh` — its hermetic test suite (R-005 branches + governance deny)
- `tests/pipeline/test_advance_all_submodules_hardening.sh` — the 2026-08-22 audit's failure-mode suite
- `tests/pipeline/test_advance_all_submodules_blast_radius.sh` — the 2026-08-23 blast-radius audit's suite
- `tests/pipeline/test_advance_all_submodules_t054.sh` — the 2026-08-26 T054 review's suite (74 cases: the three blockers, condition (C), condition (E), the vacuous passes, the re-review's five items and three nits, the round-3 review's six items and four escapes (T17–T26), the round-4 review's ten items plus the tenth escape (T27–T39), and the round-5 review's two flagless findings plus the four defects found while fixing them (T40–T49). **After the 2026-08-26 removal of R-005 step 6, every case that exercised the publish path was RETARGETED rather than deleted** — each now asserts the route is unreachable (the upstream is byte-identical in refs *and* object count; the removed flag exits 2; an unclean tree refuses with no opt-in), and three gained a second half running the same adversarial payload against a CLEAN submodule where it genuinely executes and still cannot reach the upstream. It counts **assertions executed**, not cases entered; each guarded case must assert a real minimum inside its own window; and the suite-wide assertion floor is raised whenever cases are added — it went **355 → 405** across the removal, because a removal that only ever lowers the floor is a removal nobody is checking)
- `specs/002-build-test-distribute-pipeline/research.md` — R-005 (the 7-step ordering this implements)
- `specs/002-build-test-distribute-pipeline/data-model.md` — the Submodule Advance Record entity
- `specs/002-build-test-distribute-pipeline/contracts/submodule-advance-record.schema.json` — the record schema
- `scripts/pipeline/lib/evidence.sh` — supplies the record-filename sanitizer this script reuses
- `scripts/pipeline/phase-07-closure.sh` — the caller (T055); owns parent-repo commit/push and `report.json` aggregation
- Lava `CLAUDE.md` §6.T.3 (no force push), §6.W (2-mirror scope), §6.U (no privilege escalation)